

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri
Veritabanı Güvenliği ve Şifreleme Çalışması

Github -> [https://github.com/GunIsmaail/BLM4522-](https://github.com/GunIsmaail/BLM4522-AgTabanlıParalelDagitimSistemleri)

[AgTabanlıParalelDagitimSistemleri](https://github.com/GunIsmaail/BLM4522-AgTabanlıParalelDagitimSistemleri)

<https://drive.google.com/drive/folders/1R1jtsEKA4ZjoR1mW4MuuvuEqVzCCSFT->

İSMAİL GÜN – 22290280
07.06.2026

İçindekiler

1.Projenin Amacı	2
2.Transparent Data Encryption (TDE) Uygulaması	2
2.1 TDE Kavramı ve Güvenlik Altyapısı	2
2.2 Kriptografik Anahtar Hiyerarşisi ve TDE Kurulumu	3
2.3 Şifreleme Durumunun Doğrulanması (Verification)	3
2.4 Kritik Sertifika ve Özel Anahtar (Private Key) Yedekleme Yönetimi	4
3.VERİTABANI DENETİMİ VE İZLEME (AUDIT LOGGING).....	5
3.1 Veritabanı Denetimi (Audit) Mantığı	5
3.2 Server ve Database Seviyesinde Audit Yapılandırması.....	5
3.3 DML İşlemleri Üzerinden Audit Testi ve Log Analizi	5
4. SQL Injection ve Savunma Yöntemleri	6
4.1 SQL Injection Nedir ve Nasıl Gerçekleşir?.....	6
4.2 Parametrik Sorgular (Parameterized Queries) ile Savunma Mimarisi	6
4.3 Stored Procedure Kullanarak Ön Derimli Koruma Sağlama	7
4.4 Güvenli Sorgu Test Çıktıları	7
5. Kurumsal Veritabanı Güvenlik Matrisi	8
6. Gerçek Hayattaki Kullanım Alanları	8
7. SONUÇLAR	9

1.Projenin Amacı

Bu projede Microsoft SQL Server ortamında barındırılan ECommerceDB üretim veritabanının siber tehditlere, veri sızıntılarına ve yetkisiz erişimlere karşı güvenliğini sağlamak amacıyla üç katmanlı bir savunma derinliği (Defense-in-Depth) mimarisi tasarlanmış ve uygulanmıştır. Dijital varlıkların ve hassas kullanıcı verilerinin korunmasının yasal bir zorunluluk haline geldiği günümüzde, veri güvenliği bir veritabanı mimarının en öncelikli sorumluluğudur.

Proje kapsamında gerçekleştirilen temel stratejik hedefler şunlardır:

- Veritabanı dosyalarının fiziksel olarak çalınması veya sızdırılması riskine karşı **Transparent Data Encryption (TDE)** altyapısını kurarak durağan veriyi (Data-at-Rest) koruma altına almak.
- Veritabanı üzerinde gerçekleşen tüm kritik veri hareketlerini (DML ve DDL sorgularını) izlemek ve yasal uyumluluk sağlamak amacıyla **Audit Logging** mekanizmasını yapılandırmak.
- Uygulama katmanından gelebilecek en tehlikeli web zafiyetlerinden biri olan **SQL Injection (SQLi)** saldırı vektörlerini parametrik sorgular ve saklı yordamlar (Stored Procedures) kullanarak tamamen bertaraf etmek.

2.Transparent Data Encryption (TDE) Uygulaması

2.1 TDE Kavramı ve Güvenlik Altyapısı

Transparent Data Encryption (TDE), veritabanına ait fiziksel veri dosyalarını (.mdf), log dosyalarını (.ldf) ve yedekleme arşivlerini (.bak) işletim sistemi ve disk katmanında gerçek zamanlı (real-time) olarak şifreleyen bir teknolojidir. Bu şifreleme işlemi uygulama katmanından tamamen bağımsız (şeffaf) yürüdüğü için yazılımlarda herhangi bir kod değişikliği gerektirmez. Sunucunun fiziksel disklerinin çalınması veya ham dosyaların kopyalanması durumunda, şifre çözme anahtarları olmadan verilerin okunması imkansızdır.

2.2 Kriptografik Anahtar Hiyerarşisi ve TDE Kurulumu

SQL Server'da TDE mimarisi hiyerarşik bir güvenliğe dayanır. Kurulum sırasıyla Master Key, Server Certificate, Database Encryption Key (DEK) katmanları oluşturularak tamamlanmıştır:

```
-- =====
-- SEÇİM 1: CERTIFICATE YEDEKLE
-- =====

USE master;
GO

BACKUP CERTIFICATE TDE_Cert
TO FILE = 'C:\SQLBackup\TDE_Cert.cer'
WITH PRIVATE KEY (
    FILE = 'C:\SQLBackup\TDE_Cert.pvk',
    ENCRYPTION BY PASSWORD = 'GucluSifre123!@#'
);
GO

PRINT 'Certificate basariyla yedeklendi!';
GO

-- =====
-- SEÇİM 2: TDE DURUMUNU KONTROL ET
-- =====

SELECT
    d.name AS [Veritabani],
    d.is_encrypted AS [Encrypted],
    k.encryption_state AS [Encryption State]
FROM sys.databases d
LEFT JOIN sys.dm_database_encryption_keys k
    ON d.database_id = k.database_id
WHERE d.name = 'ECommerceDB';
GO
```

2.3 Şifreleme Durumunun Doğrulanması (Verification)

Şifreleme motorunun arka planda süreci tamamlayıp tamamlamadığını denetlemek amacıyla dinamik yönetim görünümleri (DMV) üzerinden veritabanı durum analizi gerçekleştirilmiştir:

```
-- =====
-- SEÇİM 2: TDE DURUMUNU KONTROL ET
-- =====

SELECT
    d.name AS [Veritabani],
    d.is_encrypted AS [Encrypted],
    k.encryption_state AS [Encryption State]
FROM sys.databases d
LEFT JOIN sys.dm_database_encryption_keys k
    ON d.database_id = k.database_id
WHERE d.name = 'ECommerceDB';
GO
```

Sorgu sonucunda is_encrypted = 1 ve encryption_state = 3 değerleri alınmıştır. 3 kodu, veritabanının tamamen kriptografik koruma altına alındığını (Encrypted) doğrulamaktadır.

2.4 Kritik Sertifika ve Özel Anahtar (Private Key) Yedekleme Yönetimi

TDE ile şifrelenmiş bir veritabanının kurulu olduğu sunucu çöktüğünde veya veritabanı farklı bir sunucuya taşınmak istendiğinde, sunucu seviyesindeki sertifika olmadan veritabanı asla ayağa kaldırılamaz.

Bu felaket senaryosuna hazırlıklı olmak adına sertifika ve özel anahtarı (Private Key) güvenli bir şifreyle dışarı aktarılmıştır:

```
-- 1. Server Audit oluştur
CREATE SERVER AUDIT ECommerceDB_Audit
TO FILE (FILEPATH = 'C:\SQLAudit\')
WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
GO

-- 2. Audit'i etkinleştir
ALTER SERVER AUDIT ECommerceDB_Audit WITH (STATE = ON);
GO
```

 SQL2025	09/06/2026 16:46	Dosya klasörü
 SQLAudit	09/06/2026 19:13	Dosya klasörü
 SQLBackup	09/06/2026 19:11	Dosya klasörü

3. VERİTABANI DENETİMİ VE İZLEME (AUDIT LOGGING)

3.1 Veritabanı Denetimi (Audit) Mantığı

Audit Logging, veritabanı üzerinde gerçekleştirilen kritik aktivitelerin (kim, ne zaman, hangi bilgisayardan, hangi veriyi sorguladı veya değiştirdi?) izini sürmeyi sağlayan bir denetim mekanizmasıdır. Özellikle kötü niyetli iç tehditlerin (insider threats) tespiti ve yasal regülasyon uyumlulukları için bu kayıtların silinemez bir yapıda tutulması gerekir.

3.2 Server ve Database Seviyesinde Audit Yapılandırması

Denetim izlerinin işletim sistemi katmanında güvenli bir dizine yazılması ve dbo şemasındaki veri manipülasyon işlemlerinin (DML) yakalanması için gerekli mimari kurulmuştur:

```
-- 2. Audit'ı etkinleştir
ALTER SERVER AUDIT ECommerceDB_Audit WITH (STATE = ON);
GO

-- 3. Database Audit Specification oluştur
USE ECommerceDB;
GO

CREATE DATABASE AUDIT SPECIFICATION ECommerceDB_Spec
FOR SERVER AUDIT ECommerceDB_Audit
ADD (SELECT ON SCHEMA::dbo BY public),
ADD (INSERT ON SCHEMA::dbo BY public),
ADD (UPDATE ON SCHEMA::dbo BY public),
ADD (DELETE ON SCHEMA::dbo BY public)
WITH (STATE = ON);
GO

PRINT 'Audit basariyla olusturuldu!';
GO
```

3.3 DML İşlemleri Üzerinden Audit Testi ve Log Analizi

Sistemin çalıştığını doğrulamak adına Musteriler tablosuna test amaçlı bir INSERT sorgusu gönderilmiş ve oluşan log dosyası sys.fn_get_audit_file fonksiyonu ile çözümlenmiştir:

Sonuçlar

İletiler

	Tarih	Zaman	Kullanici	Islem	SQL_Komutu
1	2026-06-09	2026-06-09 16:14:14.9981964	DESKTOP-9J00M0P\cheng	SL	SELECT MusteriID, Ad, Soyad, Sehir FROM Musteri...
2	2026-06-09	2026-06-09 16:14:14.9664174	DESKTOP-9J00M0P\cheng	SL	SELECT MusteriID, Ad, Soyad, Email, Sehir FROM...
3	2026-06-09	2026-06-09 16:14:14.9291472	DESKTOP-9J00M0P\cheng	SL	SELECT MusteriID, Ad, Soyad, Email, Sehir FROM ...
4	2026-06-09	2026-06-09 16:13:20.0898959	DESKTOP-9J00M0P\cheng	IN	INSERT INTO Musteriler (Ad, Soyad, Email, Sehir) V...

4. SQL Injection ve Savunma Yöntemleri

4.1 SQL Injection Nedir ve Nasıl Gerçekleşir?

SQL Injection (SQLi), kullanıcıdan alınan girdi verilerinin (input) uygulama katmanında hiçbir kontrolden veya temizleme işleminden geçirilmeden, dinamik SQL cümleleri ile uç uca eklenmesi (string concatenation) sonucu ortaya çıkan en ölümcül veritabanı zafiyetidir.

Saldırgan, e-posta alanına test@mail.com' OR '1'='1 girdiğinde arka planda oluşan sorgu şu hale gelir:

```
CREATE PROCEDURE sp_GetMusteri
    @email NVARCHAR(100)
AS
BEGIN
    SELECT MusteriID, Ad, Soyad, Email, Sehir
    FROM Musteriler
    WHERE Email = @email;
END;
GO

-- TEST: Stored Procedure'ü çalıştır
EXEC sp_GetMusteri @email = 'test@mail.com';
GO
```

4.2 Parametrik Sorgular (Parameterized Queries) ile Savunma Mimarisi

SQL Injection saldırılarını engellemenin en temel ve etkili yolu sorguyu parametrik hale getirmektir. Parametrik sorgularda SQL komut yapısı ile kullanıcının girdisi olan veri bloğu tamamen ayrı katmanlarda işlenir. SQL motoru, girdi içerisindeki zararlı komutları çalıştırmak yerine doğrudan düz bir metin (literal string) olarak kabul eder.

```
DECLARE @search NVARCHAR(100) = '%Ankara%'

SELECT MusteriID, Ad, Soyad, Sehir
FROM Musteriler
WHERE Sehir LIKE @search;
GO

PRINT 'SQL Injection koruması: Parameterized queries başarıyla testi yapıldi!';
GO
```

4.3 Stored Procedure Kullanarak Ön Derimli Koruma Sağlama

Veritabanı güvenliğini en üst düzeye çıkarmak için tüm veri erişim süreçleri saklı yordamlar (Stored Procedure) arkasına izole edilmiştir. Stored Procedure'ler SQL Server tarafından önceden derlendiği ve yürütme planı (execution plan) hazırlandığı için gelen parametrelerin sorgu yapısını değiştirmesi imkansızdır.

```
CREATE PROCEDURE sp_GetMusteriDetails
    @email NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT MusteriID, Ad, Soyad, Email, Sehir
    FROM Musteriler
    WHERE Email = @email;
END;
GO

-- Saklı Yordamın Güvenli Şekilde Tetiklenmesi
EXEC sp_GetMusteriDetails @email = 'test@mail.com';
GO
```

4.4 Güvenli Sorgu Test Çıktıları

Geliştirilen parametrik altyapı sayesinde, Ankara lokasyonundaki 5.004 adet kurumsal müşteri kaydı, hiçbir enjeksiyon riskine maruz kalmadan milisaniyeler içerisinde başarıyla ve güvenle listelenmiştir.

	MusteriID	Ad	Soyad	Sehir
352	3520	Ad_3520	Soyad_3520	Ankara
353	3530	Ad_3530	Soyad_3530	Ankara
354	3540	Ad_3540	Soyad_3540	Ankara
355	3550	Ad_3550	Soyad_3550	Ankara
356	3560	Ad_3560	Soyad_3560	Ankara
357	3570	Ad_3570	Soyad_3570	Ankara
358	3580	Ad_3580	Soyad_3580	Ankara
359	3590	Ad_3590	Soyad_3590	Ankara
360	3600	Ad_3600	Soyad_3600	Ankara
361	3610	Ad_3610	Soyad_3610	Ankara
362	3620	Ad_3620	Soyad_3620	Ankara
363	3630	Ad_3630	Soyad_3630	Ankara
364	3640	Ad_3640	Soyad_3640	Ankara
...	...	...	...	...

Sorgu başarıyla yürütüldü. | localhost (17.0 RTM) | DESKTOP-9J0OM0P\cheng ... | ECommerceDB | 00:00:00 | Satır: 60, Sütun: 4 | 5,004 satır

5. Kurumsal Veritabanı Güvenlik Matrisi

Proje kapsamında ECommerceDB üzerinde yapılandırılan güvenlik katmanlarının operasyonel özet tablosu aşağıda sunulmuştur:

	Sütun	Durum
1	MusteriID	ŞİFRELI DEĞİL
2	Ad	ŞİFRELI DEĞİL
3	Soyad	ŞİFRELI DEĞİL
4	Email	ŞİFRELI DEĞİL
5	Sehir	ŞİFRELI DEĞİL
6	KayıtTarihi	ŞİFRELI DEĞİL

	Güvenlik Özelliği	Durum
1	Audit Logging	Etkin
2	Role-Based Access Control	Etkin
3	SQL Injection Koruması (...)	Etkin
4	TDE (Veritabanı Şifrelem...	Etkin

6. Gerçek Hayattaki Kullanım Alanları

Veritabanı şifrelemesi ve denetimi, günümüz kurumsal dünyasında isteğe bağlı bir tercih değil, yasal bir zorunluluktur:

- **KVKK ve GDPR Uyumluluğu:** Kişisel Verilerin Korunması Kanunu (KVKK) uyarınca, kullanıcıların e-posta, adres ve kimlik bilgileri gibi hassas verilerinin diskte şifreli tutulması (TDE) ve bu verilere kimlerin eriştiğinin izlenmesi (Audit) yasal bir yükümlülüktür. Aksine durumlar devasa idari para cezalarına yol açar.
- **Finans ve Sektörel Standartlar (PCI-DSS):** Kredi kartı ve ödeme altyapısı barındıran e-ticaret siteleri ile bankacılık sistemleri, PCI-DSS standartları gereği tüm veritabanı işlemlerini anlık loglamak ve SQLi açıklarına karşı parametrik mimariler kullanmak zorundadır.
- **Sağlık Sektörü (HIPAA):** Hastane otomasyon sistemlerinde (HBYS) hasta mahremiyetini korumak amacıyla tahlil, teşhis ve kişisel sağlık verileri ancak bu projede uyguladığımız AES-256 tabanlı TDE katmanları arkasında saklanabilmektedir.

7. SONUÇLAR

Proje 3 kapsamında başarıyla tamamlanan ve güvence altına alınan mühendislik görevleri şunlardır:

- **Fiziksel Güvenlik Sağlandı:** TDE entegrasyonu ile ECommerceDB donanımsal sızıntılara karşı tamamen kapatılmış, ilişkili şifreleme sertifikası ve private key anahtarı acil durum senaryoları için başarıyla yedeklenmiştir.
- **Geriye Dönük İzlenebilirlik Kuruldu:** Canlı sistemdeki tüm kritik veri manipülasyonları silinemez denetim loglarına bağlanarak kurumsal şeffaflık sağlanmıştır.
- **Kod Enjeksiyon Tehditleri Sıfırlandı:** Dinamik sorgu birleştirmeleri tamamen terk edilerek parametrik sorgu ve Stored Procedure standardına geçilmiş, yazılımsal zafiyet olasılıkları ortadan kaldırılmıştır.

Sonuç olarak; ağ tabanlı parallel ve dağıtık sistem mimarilerinde verinin sadece hızlı işlenmesi değil, aynı zamanda en yüksek standartlarda korunması gerektiği, endüstri standardı siber savunma teknikleri (MSSQL Security Suite) kullanılarak başarıyla kanıtlanmıştır.